

Lecture 1 - Basic Concepts

Intro to Mobile: Native vs Hybrid vs Cross-Platform with Flutter

Fettah KIRAN

Department of AI and Data Engineering

Yıldız Technical University

fkiran@yildiz.edu.tr

Overview

1. Introduction & Icebreaker
2. Mobile App Types: Native, Hybrid, Cross-Platform
3. Why Flutter for MTM4681?
4. Flutter Architecture: Framework, Engine, Embedder
5. Widgets: Everything is a Widget
6. Live Demo: Counter App + Hot Reload
7. Installation, Tools and Prerequisites — Windows & macOS
8. Semester Roadmap: Weeks 1-7 + 9-13, Week 8 Midterm

1. Introduction & Icebreaker

First Contact with Mobile

- **Question:** Which mobile apps have you used since you woke up today — and which do you think are Native?
 - **Examples:** Banking app, YouTube, Instagram, e-Devlet, YTU Mobile / USIS.
-
- Today: map those apps to **Native vs Hybrid vs Cross-Platform**
 - Goal by end of class: explain why we chose **Flutter + Dart** for this course

2. Mobile App Types

- **Native:** Swift/SwiftUI (iOS), Kotlin/Jetpack Compose (Android)
 - Best performance, full API access. Cost: two codebases, two teams.
- **Hybrid / WebView:** Ionic, Cordova, PWA wrapped in WebView
 - HTML/CSS/JS reused. Cost: UI lag, limited native feel.
- **Cross-Platform:** Flutter, React Native, .NET MAUI
 - One language, near-native output. Compromise we accept.

Native vs Hybrid vs Cross-Platform

	Native	Hybrid (WebView)	Cross-Platform (Flutter)
Language	Swift / Kotlin	HTML/CSS/JS	Dart
UI	Truly native	Web page in shell	Own rendering (Skia/Impeller)
Performance	Best	Weakest	Near-native
Code reuse	~0%	~90%	~90-95%
Access to GPS, sensors	Full	Plugin-limited	Full via plugins

Takeaway: For one-semester course with one team building for both stores → Cross-Platform wins.

3. Why Flutter for MTM4681?

- **Single codebase:** iOS + Android + Web + Desktop from one Dart project
- **Performance:** Compiles to ARM/x64 native, no JS bridge (vs React Native)
- **Productivity:** Hot Reload, rich Material/Cupertino widgets
- **Market:** Used by BMW, Alibaba, Google Pay, eBay Motors — good for jobs
- **Course fit:** Weeks 2-3 Dart OOP → Weeks 4-5 UI/Navigation → Weeks 8-9 persistence/Firebase all map to Flutter

4. Flutter Architecture

- **Framework (Dart):** Material, Cupertino, Widgets, Animation, Painting
- **Engine (C++):** Skia / Impeller rendering, text layout, Dart runtime
- **Embedder (Platform-specific):** Android, iOS, Web, Windows glue code

Your Dart Code (Widgets)

↓

Flutter Framework

↓

Flutter Engine (Skia/Impeller + Dart VM)

↓

Embedder → Android / iOS / Web

No OEM widgets — Flutter **draws every pixel** itself. That is why UI looks identical everywhere.

5. Widgets: Everything is a Widget

- **StatelessWidget:** static, e.g. `Text`, `Icon`, `AboutScreen`
- **StatefulWidget:** dynamic, e.g. `Checkbox`, `Counter`, `Form`
- Layout widgets: `Row`, `Column`, `Stack`, `ListView`, `Container`
- Composition over inheritance:

```
MaterialApp(  
  home: Scaffold(  
    appBar: AppBar(title: Text('MTM4681')),  
    body: Center(child: Text('Hello Flutter!')),  
  ),  
)
```

Next weeks: Week 2-3 Dart OOP → Week 4 Layouts/Forms → Week 5 Navigation.

6. Live Demo (Instructor)

- `flutter create demo_app` → run on Chrome + Android emulator
- Counter app: press **+**, see `setState()` rebuild
- **Hot Reload (<1s)** vs Hot Restart: change color, text, keep state
- Open DevTools: Widget Tree — prove *everything is a widget*

No setup needed from you today — watch, then replicate as homework.

7. Installation, Tools and Prerequisites

W4 Flutter Toolchain → moved to self-study video + 15 min setup in Week 1

Common stack for everyone:

1. **Flutter SDK** (stable, ~1 GB) + **Git**
2. **VS Code** + Flutter + Dart extensions (course IDE)
3. **Android Studio** → Android SDK + Emulator (to run phone image)
4. **Chrome** (fastest first-run target, no emulator needed)
5. Verify with `flutter doctor` — zero red `[!]` is the goal

Official guide: flutter.dev/docs/get-started/install — pick Windows / macOS tab.

Prerequisites — Common + OS Differences

- **Common:** 15 GB free disk, 8 GB RAM, admin rights, Git, VS Code, Chrome, internet
- **Windows 10/11 64-bit:**
 - Enable **Developer Mode**: Settings → Privacy & Security → For developers
 - PowerShell 5+ (preinstalled), unzip to `C:\src\flutter` (no spaces, no `Program Files`)
 - For Windows-desktop builds (optional): Visual Studio 2022 Build Tools + C++ workload
- **macOS (Ventura+):**
 - **Xcode** from App Store — required for iOS Simulator (Mac only, ~7 GB)
 - `sudo xcode-select --switch /Applications/Xcode.app` + `sudo xcodebuild -runFirstLaunch`
 - Install **CocoaPods**: `sudo gem install cocoapods` (Apple Silicon: `brew install cocoapods`)
 - Unzip SDK to `~/development/flutter` (avoid iCloud-synced folders)

iOS build/run = Mac + Xcode only. Windows students: target Android + Chrome/Web.

Windows Install — Step by Step

1. Download **flutter_windows stable .zip** from flutter.dev → extract to
`C:\src\flutter`
2. Add to PATH: Search `Env` → Environment Variables → Path → Add
`C:\src\flutter\bin` → restart terminal
3. Check: `flutter --version` then `flutter doctor`
4. Accept licenses: `flutter doctor --android-licenses` (press `y` to all)
5. Install **Android Studio** → SDK Manager: Android SDK 34 + Platform-Tools + Emulator → AVD Manager: create Pixel 7 / API 34 image → boot it
6. In VS Code: Extensions → **Flutter** (auto-installs Dart) → `Ctrl+Shift+P` →
`Flutter: New Project` → `flutter run -d chrome` first, then `-d emulator-`
`5554`

macOS Install — Step by Step

This Mac) → unzip to `~/development/flutter`

2. Add to PATH (zsh): `echo 'export`

```
PATH="$HOME/development/flutter/bin:$PATH"' >> ~/.zshrc && source  
~/.zshrc
```

3. Check: `flutter --version` then `flutter doctor`

4. iOS toolchain: install **Xcode** → `sudo xcode-select --switch`

```
/Applications/Xcode.app → sudo xcodebuild -runFirstLaunch → sudo gem  
install cocoapods → open -a Simulator
```

5. Install **Android Studio** (JetBrains Toolbox or .dmg) → SDK + Emulator → create Pixel AVD

6. In VS Code: Extensions → **Flutter** → `Cmd+Shift+P` → `Flutter: New Project` → run on **Chrome** first, then iOS Simulator / Android Emulator

Tools We Use in Class: VS Code + Devices

- **VS Code extensions:** `Flutter` + `Dart` (official) — plus Error Lens (optional)
 - **Pick device in VS Code:** bottom-right status bar → `Chrome (web-javascript)` for Day 1, emulator/simulator later
 - **15-min in-class check:** `flutter doctor -v` → `flutter create hello` → `flutter run -d chrome`
 - **No laptop power?** Use **DartPad** (<https://dartpad.dev>) for Weeks 2-3 Dart OOP — zero install
-
- **Task:** Open Command Palette → `Flutter: New Project` → name it `hello` → run on Chrome. Screenshot your running Counter app.

flutter doctor — Fix Table

Android toolchain [!]	Install Android Studio SDK + flutter doctor --android-licenses	Same + set flutter config --android-sdk ~/Library/Android/sdk
Chrome [!]	Install Chrome + set CHROME_EXECUTABLE	Install Chrome, same
Visual Studio [!]	Optional — install VS Build Tools only if Windows-desktop target	Ignore (Windows-only check)
Xcode / CocoaPods [!]	Ignore (Mac-only) — target Android/Chrome	Install Xcode + sudo gem install cocoapods / brew install cocoapods
Network /	Campus Wi-Fi: enable mobile	Same

Homework Setup Checklist

- ☐ **Windows:** SDK in `C:\src\flutter`, Developer Mode ON, `flutter doctor` clean (except VS/Xcode-optional)
- ☐ **macOS:** SDK in `~/development/flutter`, Xcode + Simulator boot, CocoaPods installed
- ☐ VS Code Flutter+Dart extensions → Counter app runs on **Chrome**
- ☐ (Recommended) Boot **Android emulator** (both OS) / **iOS Simulator** (Mac) and re-run app
- ☐ Bookmark **DartPad**: <https://dartpad.dev> — fallback for Weeks 2-3
- ☐ Problem? Bring `flutter doctor -v` screenshot to Week 2 lab

Lab next week needs working SDK — Week 2 is Dart OOP I (`Matrix`, `Vector` hierarchy).

Flutter setup videos

- [Windows](#)
- [Mac](#)

8. Semester Roadmap — Fall 2026 Form

- **Weeks 1-7 (Pre-Midterm):** 1 Intro, 2 Dart OOP I, 3 Dart OOP II + Async, 4 Widgets/Layouts, 5 Navigation, 6 Networking, 7 Midterm Review
- **Week 8: MIDTERM**
- **Weeks 9-13 (Post-Midterm):** 9 Hive/sqlite, 10 Firebase Firestore + Streams, 11 Animations/CustomPaint, 12 Provider/Riverpod MVVM, 13 Device/Security/Publishing + Showcase & Finalization

OOP kept 2 weeks for accreditation even if you know it — seniors, help your team.

Practice Resource

Let's visit:

[DartPad — Try Dart & Flutter online](#)

[Flutter Codelab: Write your first app](#)

Run the Week 1 counter example in your browser tonight — zero install!

MTM4681 - Readiness Survey

https://docs.google.com/forms/d/e/1FAIpQLSeZktvCwdnPSH7dtbq_MKarB2qLdd7uT1FUzBIU4d-jBZDL-A/viewform?usp=publish-editor

Update link for Fall 2026 MTM4681 cohort — same form pattern as MTM4692.

Questions & Discussion

Thank you!

Install Flutter, run `flutter doctor`, and bring one app idea for the next class!